Practical – 4

Study and Implement Simple Linear Regression.

1. What is Linear Regression?

Linear regression is a statistical and machine learning method used to model the relationship between variables. It tries to draw a **best-fit straight line** through data points to predict values.

2. How Linear Regression works?

Core Idea:
It assumes a relationship like:

$$y = mx + c$$

Where:

- **x** → Input (independent variable)
- **y** → Output (dependent variable)
- **m** → Slope (how much y changes when x changes)
- **c** → Intercept (value of y when x = 0)

What It Does

- Learns from past data
- Finds the line that minimizes prediction error (usually using **least squares**)
- Uses that line to predict future values

Example: If you use linear regression to predict house price based on size, it learns how price increases as size increases.

Ways To Implement Linear Regression:

There are two ways to implement Linear Regression in Machine Learning

i. Normal Equation: It's a pure math approach where we use maths like statistics to find the parameters to fit the data in the line.

ii. Sklearn: It's a built-in library in python which uses Ordinary Least Squares Approach to find the parameters and fit the data.

3. Implementation of Linear Regression Using Normal Equation:

Step 1: Dataset Preparation

We are using Height and Weight data for this practical from Kaggle.

Dataset Preparation

```
import pandas as pd
import numpy as np
df=pd.read_csv(path + "/SOCR-HeightWeight.csv")
df.head()
```

	Index	Height(Inches)	Weight(Pounds)
0	1	65.78331	112.9925
1	2	71.51521	136.4873
2	3	69.39874	153.0269
3	4	68.21660	142.3354
4	5	67.78781	144.2971

Step 2: Perform Standardization on the dataset:-

Now we write the function to use normal equation to find parameters for performing linear regression.

Normal Equation

```
from numpy import sign
def intercept_slop(x,y):
    n=len(x)
    sig_x=np.sum(x)
    sig_y=np.sum(y)
    sign_xy=np.sum(x*y)
    sig_x2=np.sum(x**2)
    m = (n*sign_xy-sig_x*sig_y)/(n*sig_x2-sig_x**2)
    c = (sig_y-m*sig_x)/n
    return m,c

m,c = intercept_slop(df["Height(Inches)"],df["Weight(Pounds)"])
print(m,c)
```

```
3.0834764454016637 -82.57574306445234
```

Step 3: Prediction:

Now we do the prediction of the weights for the corresponding height.

Prediction

```
def predict(x,m,c):
    y_pred=m*x+c
    y_pred=pd.Series(y_pred,name="Weight(Pounds)")
    return y_pred
y_pred=predict(df["Height(Inches)"],m,c)
y_pred.head()
```

	Weight(Pounds)
0	120.265544
1	137.939722
2	131.413637
3	127.768536
4	126.446372

dtype: float64

Step 4: Mean Squared Error:

Now let us calculate the mean squared error from sklearn.metrics library.

Mean Squared Error

```
from sklearn.metrics import r2_score, mean_squared_error
y_org = df['Weight(Pounds)']
print('Mean Squared Error:', mean_squared_error(y_pred, y_org))
```

```
Mean Squared Error: 101.58853248632849
```

4. Implementation of Linear Regression Using sklearn:

Step 1: Import the Libraries

Linear Regression Using sklearn

```
from sklearn.linear_model import LinearRegression

from sklearn.metrics import r2_score, mean_squared_error
from sklearn.model_selection import train_test_split
```

Step 2: Load The Dataset

We are using the same dataset as the one used for normal equation.

Load The Dataset

```
X = df[["Height(Inches)"]]
y = df["Weight(Pounds)"]
```

Step 3: Data Splitting and Model Training

Data Splitting and Model Training

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
```

```
LinearRegression
LinearRegression()
```

Step 4: Prediction

Prediction

```
y_pred = model.predict(X_test)
y_pred
```

```
array([128.41144971, 126.79032375, 126.06117947, ..., 128.86141782,
       124.33022578, 135.00025306])
```

Step 5: Error Calculation

Mean Squared Error

```
from sklearn.metrics import mean_squared_error
print("Mean Squared Error:", mean_squared_error(y_pred, y_test))
```

```
Mean Squared Error: 102.48790963792534
```